Kubernetes Scheduling

requests, limits, placement and disruption budgets

Watch the 4-min video

> **In one line:** the scheduler places pods by their **requests**, the kernel enforces **limits** (CPU throttles, memory OOM-kills), QoS classes decide who is evicted first, and **affinity, taints, topology spread** and **disruption budgets** keep replicas where they should be and alive through maintenance.

1 Three symptoms, one cause

Your cluster looks half empty, yet a new pod has been **Pending** for an hour. Another service keeps restarting with **OOMKilled**. And during a routine node upgrade, all three replicas of your API went down together. All three come from how Kubernetes schedules pods and manages resources, and all three are fixed in the pod spec.

2 How the scheduler picks a node

For each unscheduled pod, kube-scheduler runs two phases. **Filtering** removes nodes that can't run it: not enough unreserved CPU or memory, a taint the pod doesn't tolerate, a required node affinity that doesn't match, a volume in another zone, a spread constraint that would be violated. **Scoring** ranks the rest (spreading, preferred affinities, image already present, balanced utilisation), and the pod is bound to the best one. If no node passes filtering, the pod stays Pending and the event message lists the reasons per node. If a higher-priority pod can't fit, the scheduler can **preempt** lower-priority pods (PriorityClass), and the cluster autoscaler can add nodes (lesson 58).

3 Requests and limits

Each container may declare, for CPU and memory, a **request** and a **limit**. CPU is in cores or millicores (`500m` = half a core), memory in bytes (`512Mi`). **Requests are reservations**: the scheduler only places a pod where the sum of requests fits within the node's **allocatable** capacity (node capacity minus what's reserved for the OS and kubelet). Actual usage is irrelevant to placement. **Limits are caps** enforced at runtime by cgroups (lesson 55).

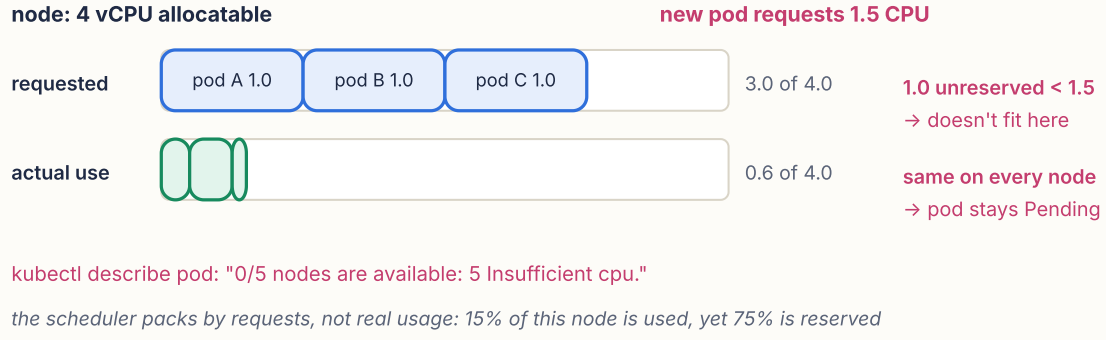


Figure 1. Why a pod is Pending on a quiet cluster: over-sized requests reserve capacity that nobody uses.

So set requests from measured usage (for example the p90–p95 of real consumption plus headroom), not guesses. Requests that are too high waste money and cause Pending pods; too low, and nodes are over-

committed, leading to throttling, evictions and noisy neighbours.

4 Over the limit: throttling vs OOMKilled

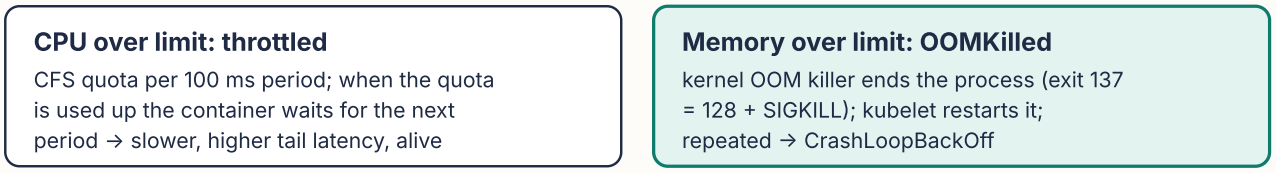


Figure 2. CPU is compressible, memory is not.

CPU is compressible. A CPU limit becomes a CFS quota: with a 1-CPU limit the container may use 100 ms of CPU time per 100 ms period, across all its threads. A multi-threaded app can burn its quota in 25 ms and then sit idle for 75 ms, which shows up as latency spikes rather than errors. Many teams therefore set CPU **requests** but no CPU limit for latency-sensitive services, letting them use idle CPU while requests still guarantee their share.

Memory is not compressible. A container that exceeds its memory limit is killed by the kernel's OOM killer, shown as `OOMKilled` with exit code 137, and restarted. Common causes: a JVM or Node heap sized larger than the container limit, unbounded caches, or a memory leak. The usual advice is **memory limit = memory request**, so what you reserve is what you can use.

5 QoS classes and eviction

QoS class	Rule	Under node memory pressure
Guaranteed	every container has CPU and memory requests = limits	evicted last; lowest OOM score
Burstable	at least one request or limit set, not Guaranteed	evicted when using more than requested
BestEffort	no requests or limits at all	evicted first

When a node runs short of memory or disk, the kubelet **evicts** pods to protect the node. It ranks candidates by whether their usage exceeds their requests, then by pod priority, then by how far over their requests they are, which in practice follows the QoS order above. Critical services should be Guaranteed, or at least Burstable with realistic memory requests; BestEffort is for work you don't mind losing.

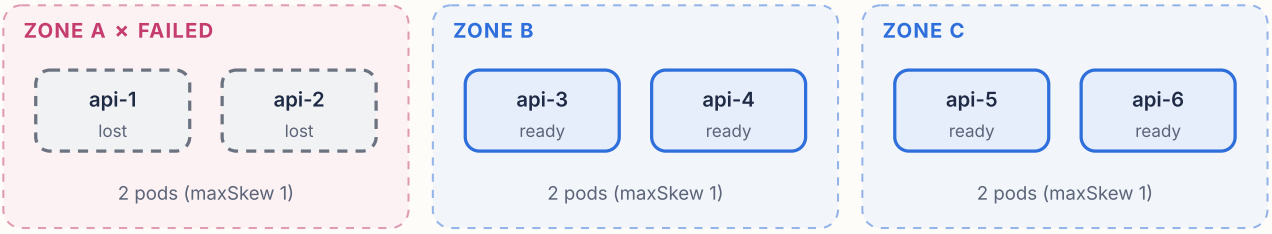
6 Steering placement

- **Node affinity**: require or prefer nodes with labels, e.g. `requiredDuringSchedulingIgnoredDuringExecution` on `accelerator=nvidia`. **Pod anti-affinity** keeps replicas off the same node.
- **Taints and tolerations**: a taint on a node repels pods unless they tolerate it, the reverse of affinity. Effects: `NoSchedule`, `PreferNoSchedule`, and `NoExecute` (also evicts running pods). Taint GPU nodes so only GPU workloads land there; give those workloads both a toleration and an affinity, since a toleration only allows, it doesn't attract.
- **Topology spread constraints**: spread replicas evenly over a topology key such as `topology.kubernetes.io/zone` or `kubernetes.io/hostname`. `maxSkew: 1` means no zone may have

more than one pod more than the emptiest zone; `whenUnsatisfiable` chooses between refusing to schedule (`DoNotSchedule`) and best effort (`ScheduleAnyway`).

7 Pod disruption budgets

Voluntary disruptions are ones the cluster chooses: node drains for upgrades, autoscaler scale-down. A **PodDisruptionBudget** limits them: with `minAvailable: 2` for a 3-replica service, a drain evicts one pod, waits until its replacement is ready elsewhere, and only then evicts the next. The eviction API refuses requests that would break the budget, so the upgrade slows down instead of causing an outage. PDBs don't protect against **involuntary** disruptions such as a node crash or a zone outage; that's what spreading is for. Avoid budgets that can never be satisfied (`minAvailable` equal to replicas), which block node drains forever.



6 replicas, topologySpreadConstraints on topology.kubernetes.io/zone → 2 / 2 / 2

zone a fails → 4 ready pods keep serving; PDB minAvailable 4 stops a drain taking more

capacity rule: the 4 survivors (2 zones) must carry peak load on their own

Figure 3. Spread plus budget: spreading survives involuntary failures; the PDB protects against voluntary ones.

```
spec:
  replicas: 6
  template:
    spec:
      topologySpreadConstraints:
      - maxSkew: 1
        topologyKey: topology.kubernetes.io/zone
        whenUnsatisfiable: DoNotSchedule      # or ScheduleAnyway
        labelSelector: {matchLabels: {app: api}}
      containers:
      - name: api
        resources:
          requests: {cpu: 500m, memory: 512Mi}
          limits:   {memory: 512Mi}           # memory limit = request
---
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata: {name: api}
spec: {minAvailable: 4, selector: {matchLabels: {app: api}}}
```

Figure 4. The worked example's spec: zone spread, realistic requests, memory limit equal to request, and a budget of 4.

WORKED EXAMPLE: SIX REPLICAS ACROSS THREE ZONES

The API Deployment has 6 replicas with a zone spread of `maxSkew: 1`, so the scheduler places 2 pods in each of zones a, b and c. A PodDisruptionBudget keeps at least 4 available, so node upgrades drain at most 2 pods at a time. When zone a fails, its 2 pods are lost and 4 keep serving, sized to carry peak load alone. Watch the constraint during the outage: with `DoNotSchedule`, replacements may be blocked from piling into zones b and c while zone a still counts as a domain, so teams either accept running on 4 until the zone returns, or use `ScheduleAnyway` to favour recovery over perfect balance.

COMMON MISTAKES

- Copy-pasted requests far above real usage: Pending pods and wasted nodes.
- No memory requests (BestEffort) on important services.
- JVM or runtime heap larger than the container memory limit.
- No PDB, or a PDB with minAvailable equal to replicas that blocks every drain.

8 Key terms

Request: reserved CPU/memory used for scheduling

Limit: runtime cap: CPU throttles, memory OOM-kills

QoS class: Guaranteed, Burstable, BestEffort

Taint / toleration: node repels pods unless they tolerate it

Topology spread: even distribution across zones or nodes

PodDisruptionBudget: limit on voluntary evictions

9 Interview questions

Q1. What happens when a pod exceeds its memory limit? BASIC

The container is OOMKilled (exit 137) and restarted; exceeding a CPU limit only throttles it.

Q2. How do you keep replicas available? BASIC

Realistic requests, topology spread across zones (and nodes), and pod disruption budgets during maintenance.

Q3. Requests vs limits? BASIC

Requests are what the scheduler reserves and guarantees; limits are the maximum a container may use, enforced by cgroups.

Q4. A pod is Pending though nodes show 20% CPU usage. Why? SCENARIO

Scheduling uses requests, not usage: the nodes' allocatable capacity is already reserved by other pods' requests. Right-size requests or add nodes.

Q5. A latency-sensitive service shows p99 spikes but low average CPU. What do you suspect? SCENARIO

CFS throttling from a CPU limit: bursts use up the quota early in each 100 ms period. Check throttling metrics; raise or remove the CPU limit and keep requests.

Q6. Does a PodDisruptionBudget protect against a zone outage? TRICKY

No, PDBs only govern voluntary evictions through the eviction API; zone failures are involuntary, so you need spreading and spare capacity.

Q7. You tainted GPU nodes and added tolerations to GPU pods. Why do GPU pods still land on CPU nodes? TRICKY

A toleration only permits scheduling on tainted nodes; add node affinity or a nodeSelector to require GPU nodes.

Next: SD-V-008 · Kubernetes Autoscaling backendroadmap.com